Relatório Experimental: Soft Actor-Critic (SAC)

Aluno: Pedro Reis | **Ambiente:** Hopper-v5 (100.000 passos)
Fork GitHub (algorithms/sac.py e configs): https://github.com/Pedro-Reis2/t-zero-learning
Projeto Weights & Biases (todos os runs públicos): https://wandb.ai/pedroreis2-federal-university-of-goi-s/sac-assignment

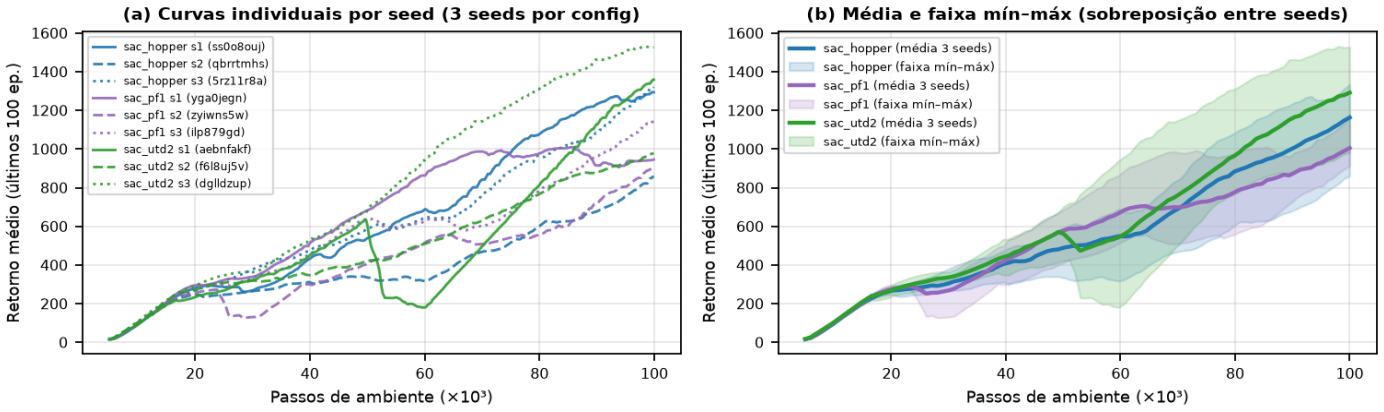
Implementação e Verificação (Partes 1 e 2): As funções squashed_gaussian_sample (algorithms/sac.py, linhas 145–155, amostragem reparametrizada via rsample() e correção jacobiana do tanh) e compute_soft_q_target (linhas 179–180, alvo pessimista $r + \gamma(1 - d)(\min(Q'_1, Q'_2) - \alpha \log \pi(a'|s'))$) passaram nos 16 testes de tests/test_sac.py. No sanity check sac_pendulum (run zexmnmhh, 20k passos), o agente obteve eval/mean_return = -165.5 ± 51.6 e eval/mean_return_sampled = -150.3 ± 72.5 (> -200).

Parte 3 — Eficiência Amostral no Hopper-v5 (100 mil passos, 3 seeds por config)

Partindo de configs/sac_hopper.yml (sac_hopper), avaliamos 4 candidatos com 3 seeds (seed=1, 2, 3) e orçamento exato de 100.000 passos. A tabela abaixo resume cada mudança, o motivo observado citando o run específico no W&B, as métricas finais como **média (mín–máx)** e o veredito:

Configuração (Runs no W&B)	Mudança (antigo → novo)	Motivo observado (citando o run onde foi visto)	eval/mean_return média (mín–máx)	r_last100 final média (mín–máx)	Veredito
sac_hopper (Inicial/Final) ss0o8ouj, qbrrtmhs, 5rz11r8a	Padrão CleanRL (ls=5000, tau=0.005, pf=2, utd=1)	Config inicial de referência; alta variância entre seeds na avaliação final (20.2 no s3 vs 2462.5 no s2).	**1546.8** (20.2–2462.5)	**1163.7** (862.2–1334.8)	**Referência (Config Final)**
sac_ls1000 vfc8ax8e, f2vi8mh6, jy4eaw0u	learning_starts: 5000 → 1000	No run ss0o8ouj (sac_hopper s1), os primeiros 5.000 passos (5% do orçamento) foram gastos em coleta aleatória com r_last100 estagnado em ~18, motivando iniciar o treino em 1.000 passos.	**947.9** (733.9–1374.3)	**775.5** (725.3–805.1)	**Piora** (r_last100 < 862.2)
sac_tau001 11gntid3, 1z94x8bs, tux9q718	sac.tau: 0.005 → 0.01	No run qbrrtmhs (sac_hopper s2), losses/qf1_values levou ~60k passos para subir a ~170 enquanto o retorno estagnou em ~330 entre 30k–60k passos, motivando dobrar a taxa Polyak dos alvos.	**862.1** (709.2–944.4)	**818.7** (536.1–1110.1)	**Indistinguível / Piora**
sac_pf1 yga0jegn, zyiwns5w, ilp879gd	policy_frequency: 2 → 1	No run ss0o8ouj (sac_hopper s1), atualizar o ator só a cada 2 passos do crítico atrasou a descida de losses/actor_loss até ~50k passos, motivando atualizar o ator em todo passo.	**2124.9** (985.3–2988.3)	**1005.7** (913.9–1157.1)	**Indistinguível do ruído**
sac_utd2 aebnfakf, f6l8uj5v, dglldzup	updates_per_step: 1 → 2	Nos runs ss0o8ouj e qbrrtmhs, r_last100 ainda crescia quase linearmente aos 100k passos sem atingir platô com 94.900 gradientes, motivando dobrar para 2 atualizações/passo (189.800 gradientes).	**1200.2** (1166.3–1257.5)	**1292.2** (981.6–1528.1)	**Indistinguível do ruído**

Análise e Escolha do Config Final: Iniciar o treino em 1.000 passos (sac_ls1000) causou sobreajuste precoce nos poucos episódios iniciais do buffer, deixando todo o intervalo de r_last100 (725.3–805.1) abaixo do mínimo do config inicial (862.2). Aumentar tau=0.01 (sac_tau001) reduziu as médias e elevou o viés charts/q_minus_return médio de -16.9 para +6.5. O candidato sac_pf1 obteve a maior média em eval/mean_return (2124.9 vs 1546.8), mas com ampla sobreposição de faixa e média menor em r_last100 (1005.7, faixa 913.9–1157.1 totalmente contida em 862.2–1334.8). Inversamente, sac_utd2 obteve a maior média em r_last100 (1292.2), mas média menor em eval/mean_return (1200.2) e faixas sobrepostas, custando o dobro do tempo (83,3 min vs 42,1 min). Como nenhum candidato superou o ruído entre seeds, **o config final é o próprio config inicial (sac_hopper)**. A Figura abaixo mostra o config inicial/final e os dois candidatos que chegaram mais perto (sac_pf1 e sac_utd2, 3 seeds cada):

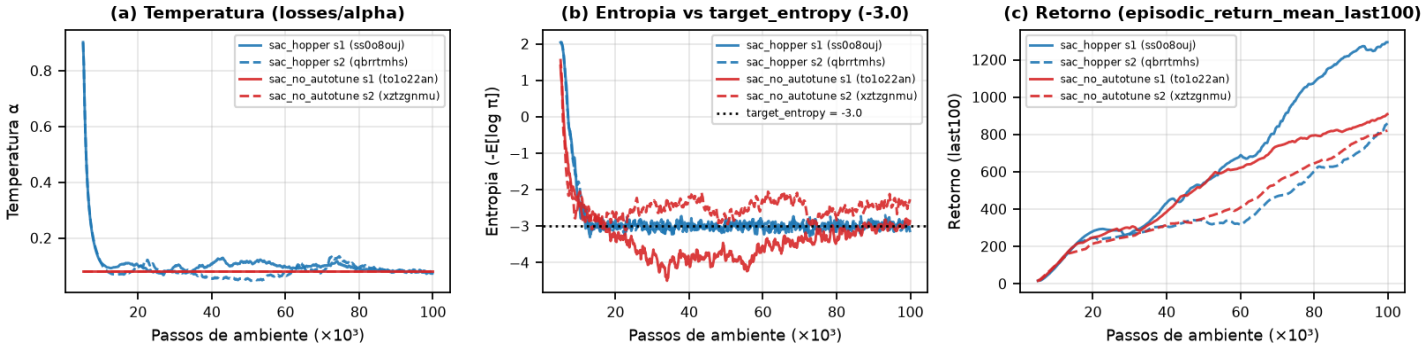


Parte 4 — O que cada peça faz (Ablações em Hopper-v5, 2 seeds cada)

Partindo do config final da Parte 3 (sac_hopper, seeds 1 e 2: ss0o8ouj e qbrrtmhs, média eval/mean_return = 2310.2 e r_last100 = 1078.2), executamos as duas ablações com 2 seeds cada.

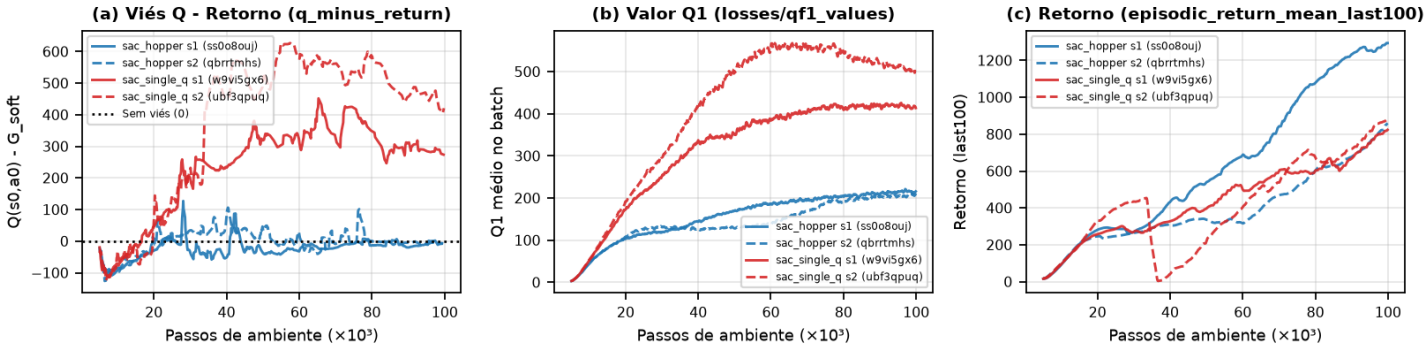
1. Ablação 1 — Sem ajuste automático da temperatura (sac.autotune=false, sac.alpha=0.08)

Escolha de sac.alpha=0.08: Nos 3 seeds de sac_hopper (ss0o8ouj, qbrrtmhs, 5rz11r8a), losses/alpha parte de exp(0) = 1.0, cai para ~0.10 aos 15k passos e oscila entre 0.06 e 0.11 no restante do treino, terminando em 0.0728, 0.0805 e 0.0750 (média 0.0761). Escolhemos portanto sac.alpha=0.08 como temperatura fixa representativa do regime estacionário.
Previsão (antes do experimento): Esperávamos que, mesmo fixando $\alpha = 0.08$ no valor assintótico, a falta de α alto nos primeiros 15k passos e a ausência de realimentação fariam losses/entropy desviar de charts/target_entropy = -3.0, prejudicando a exploração e reduzindo o retorno.
O que aconteceu: A previsão **se confirmou**. Enquanto sac_hopper (azul) ajusta losses/alpha (painel a) e mantém losses/entropy cravado em target_entropy = -3.0 dos 15k aos 100k passos (painel b), nos runs sem autotune (to1o22an e xztzgnmu, vermelho) a entropia fica sem controle: no seed 1 (to1o22an) despenca até -4.3 entre 30k–60k passos (política prematuramente determinística, eval/mean_return = 317.0), e no seed 2 (xztzgnmu) fica acima do alvo em ~-2.1 (eval/mean_return = 1087.7). A média de eval/mean_return caiu de **2310.2 para 702.4** (e r_last100 caiu de 1078.2 para 867.0).
Explicação no código (algorithms/sac.py): Na **linha 341** (self.alpha = self.log_alpha.exp().item() if cfg.autotune else float(cfg.alpha)), autotune=False congela self.alpha = 0.08, e o bloco if cfg.autotune: nas **linhas 489–497** de _update() deixa de otimizar compute_alpha_loss (**linhas 198–206**: (-log_alpha.exp() * (log_pi.detach() + target_entropy)).mean()), desativando a realimentação que aumenta α quando *-E[log π] < -3.0* e reduz α quando *-E[log π] > -3.0*.



2. Ablação 2 — Um crítico só (sac.clipped_double_q=false)

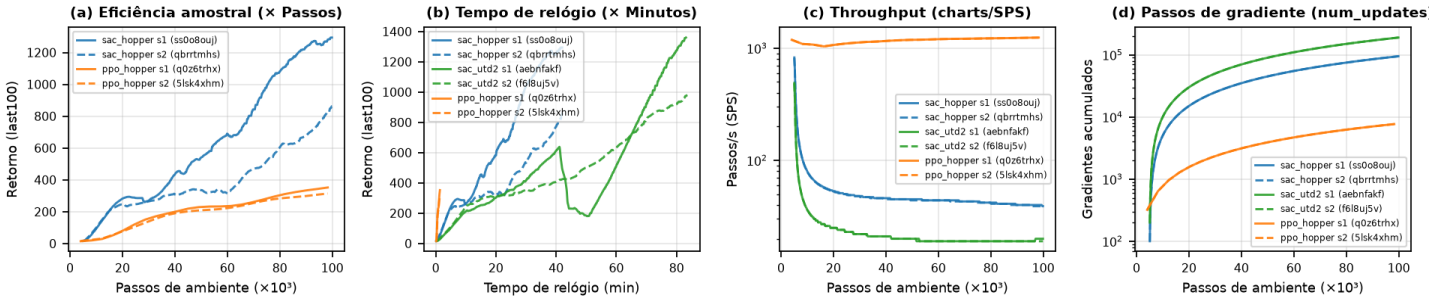
Previsão (antes do experimento): Esperávamos que usar apenas Q_1 no lugar de $min(Q_1, Q_2)$ gerasse forte viés de superestimação (charts/q_minus_return > 0 e losses/qf1_values inflados), degradando o retorno porque o ator explora picos espúrios de Q_1.
O que aconteceu: A previsão **se confirmou de forma contundente**. Na Figura abaixo, enquanto no sac_hopper (azul) o viés charts/q_minus_return permanece levemente negativo (média -27.6 em ss0o8ouj e -6.9 em qbrrtmhs, painel a) e losses/qf1_values estabiliza em ~216 (painel b), nos runs com um crítico só (w9vi5gx6 e ubf3qpuq, vermelho) charts/q_minus_return dispara para **+300 a +620** (média +171.1 e final +251.3 em w9vi5gx6; média +291.2 e final +442.2 em ubf3qpuq) e losses/qf1_values mais que dobra (409.0 e 491.4). No seed 2 (ubf3qpuq, painel c), quando a superestimação passa de +300 (~35k passos), o retorno colapsa de 450 para 10 e termina com eval/mean_return = 48.6 (média **745.7** nos 2 seeds vs **2310.2** no baseline).
Explicação no código (algorithms/sac.py): Na **linha 367** (pick_q2 = (lambda q1, q2: q2) if cfg.clipped_double_q else (lambda q1, q2: q1)), clipped_double_q=False faz pick_q2 devolver q1 no lugar de q2. Com isso, tanto o alvo na **linha 455** (compute_soft_q_target, **linha 179**: torch.min(next_q1, next_q2)) quanto a perda do ator na **linha 483** (compute_actor_loss, **linha 195**: torch.min(q1_pi, q2_pi)) calculam $min(Q_1, Q_1) = Q_1$. Como o ator move as ações na direção que maximiza $Q_1(s, a)$ e o próprio Q'_1 avalia a' no alvo de Bellman, erros positivos de aproximação se retroalimentam continuamente.



Parte 5 — SAC vs PPO Contínuo no Hopper-v5 (100 mil passos, 2 seeds)

Comparamos o melhor SAC (sac_hopper, runs ss0o8ouj e qbrrtmhs, além de sac_utd2, runs aebnfakf e f6l8uj5v, para analisar updates_per_step) com o PPO contínuo (ppo_hopper, runs q0z6trhx e 5lsk4xhm, rodados com --config ppo_pendulum --override env_id=Hopper-v5 total_timesteps=100000 ppo_continuous_action.gamma=0.99 track=true seed=1,2).

Previsão (antes do experimento): Esperávamos que, no orçamento de 100.000 passos, o SAC atingisse retorno muito superior por passo de ambiente graças ao reuso off-policy no replay buffer, mas que o PPO contínuo tivesse charts/SPS muito maior e charts/num_updates muito menor, terminando os 100k passos em poucos minutos e chegando mais longe no curtíssimo prazo de tempo de relógio.



1. Eficiência Amostral (Retorno × Passos) vs. Tempo de Relógio (Retorno × Minutos)

• **Eficiência amostral (painel a):** Confirmou a previsão. Em 100.000 passos, o PPO contínuo atinge r_last100 de apenas **361.1** (q0z6trhx) e **320.7** (5lsk4xhm), com eval/mean_return médio de **370.6** (410.3 e 331.0). Com os mesmos 100.000 passos, os seeds 1 e 2 do sac_hopper atingem r_last100 de **1294.2** e **862.2** (média **1078.2**; 1163.7 nos 3 seeds) e eval/mean_return médio de **2310.2** (2157.8 e 2462.5) — avaliação **6,2× maior**.
• **Tempo de relógio (painel b) — quem chega mais longe com o mesmo tempo?** Depende do horizonte de tempo: em **1,4 minutos** (tempo total em que o PPO conclui os 100k passos), **o PPO chega mais longe** (retorno 340.9, eval 370.6), pois em 1,4 min o sac_hopper (39 SPS) executou apenas ~3.300 passos e ainda está na coleta aleatória (learning_starts=5000, retorno ~18). Porém, após **~6 minutos** de relógio (~14k passos), o sac_hopper ultrapassa o retorno final do PPO e segue subindo até 42,1 minutos.
• **Explicação via charts/SPS (painel c) e charts/num_updates (painel d):** O PPO coleta rollouts vetorizados (num_envs=4, num_steps=1024, batch_size=4096) em 24 iterações e faz apenas 24 × 10 × 32 = **7.680 passos de gradiente**, rodando a **1.246–1.247 SPS** (1,4 min). Já o sac_hopper (updates_per_step=1) faz 1 passo de gradiente dos críticos (batch_size=256) a cada passo do ambiente após o passo 5.000, somando **94.999 atualizações dos críticos** (num_updates=94900 no log de 99.900) mais 47.499 de ator/α, reduzindo o throughput em 32× (**39–40 SPS**, 42,1 min).
• **Como updates_per_step da Parte 3 muda a resposta:** Com sac.updates_per_step=2 (sac_utd2, verde nos painéis b–d), charts/num_updates dobra para **189.800**, charts/SPS cai exatamente à metade (**19–20 SPS**) e o tempo de relógio dobra para **83,3 min**, exigindo ~12 min (o dobro do utd=1) para superar o retorno que o PPO atinge em 1,4 min.

2. Mecanismo e Estimativa de Passos de Gradiente por Transição

• **Peças do SAC que explicam as diferenças:** (1) **Replay buffer off-policy** (ReplayBuffer, linhas 212–243): retém todas as 100.000 transições (buffer_size = 10^6) para reuso contínuo, enquanto o PPO descarta o rollout de 4.096 transições ao fim de cada iteração; (2) **Ator reparametrizado + dois críticos Q** (linhas 145 e 195): atualiza o ator usando o gradiente determinístico $\nabla_\theta \, min(Q_1, Q_2)(s, a_\theta(s))$ em vez da razão de probabilidades ruidosa do PPO; (3) **Máxima entropia com α adaptativo** (linhas 179 e 206): evita colapso prematuro da política.
• **Quantas vezes, em média, cada transição entra em um passo de gradiente:**

Algoritmo / Config	Passos de gradiente (num_updates)	Amostras por passo (batch_size)	Cálculo do reuso médio por transição em 100.000 passos	Reuso médio / transição	SPS / Tempo
PPO Contínuo (ppo_hopper)	7.680 (24 iter × 10 ép. × 32 mb)	128 (minibatch)	Cada lote de 4.096 transições passa por update_epochs = 10 épocas sem reposição e é descartado (98.304 passos usados).	**10,0 vezes** (9,83 sobre 100k)	1.247 SPS **1,4 min**
SAC (utd = 1) (sac_hopper)	94.999 Q updates (+47.499 ator/α)	256	*(T - L) · u · B / T* = 95.000 × 1 × 256 / 100.000 = **243,2** (transições iniciais *t* ≤ *5k*: 256 ln(100k/5k) ≈ **766,9 vezes**).	**243,2 vezes** (até 766,9 nas iniciais)	39–40 SPS **42,1 min**
SAC (utd = 2) (sac_utd2)	189.998 Q updates (+94.999 ator/α)	256	*(T - L) · u · B / T* = 95.000 × 2 × 256 / 100.000 = **486,4** (transições iniciais *t* ≤ *5k*: 512 ln(100k/5k) ≈ **1.533,8 vezes**).	**486,4 vezes** (até 1.533,8 nas iniciais)	19–20 SPS **83,3 min**